

More Line Fitting and Reading from A File with GDP Data

Lets look at some real data and fit a line. [Life expectencay verus GDP](#) is a common graph. Let's learn about it. We are going to

- Read it in as an excel
- Save and Read it in as a csv
- fit a line to the data
- See if we like the fit
- Then for a bonus recreate the plot from the web.

In order to do this we need to start working with Pandas. I am rearranging a few classes so lets see how it goes.

Pandas is for Panel Data and it is really like reading in and working with excel sheets. It is going to take us a long time to learn all about it. Unlike Midnights or Tortured Poets Departments Pandas is too big to drop in one day. So consider this more of a start and don't worry soon you'll get better.

YOU MIGHT need to add tge library openpyxl

uv add openpyxl

```
In [1]: import pandas as pd # A new one today!
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

np.set_printoptions(legacy='1.25') #removes the funny printing
```

Step 1.

- get GDP_LifeExpectancy2018.xlsx from github
- read it in.
- in pandas you name things df. df means dataframe. I sometimes add a second part to the name like dfGDP but other times I just name df.
- then in the next cell just type df to look at it. just typing df looks nicer than print(df)
- you might need to install openpyxl (I did)

```
In [2]: df=pd.read_excel('GDP_LifeExpectancy2018.xlsx')
```

```
In [3]: df
```

```
Out [3]:
```

	Country	Code	Year	LifeExpectancy	GDP per capita	Population	Continent	PrintName
0	Afghanistan	AFG	2018	63.0810	1934.5550	36686788	Asia	NaN
1	Albania	ALB	2018	79.1838	11104.1660	2877019	Europe	NaN
2	Algeria	DZA	2018	76.0656	14228.0250	41927008	Africa	NaN
3	Angola	AGO	2018	62.1438	7771.4420	31273538	Africa	NaN
4	Argentina	ARG	2018	76.9994	18556.3830	44413592	South America	NaN
...	...	...	...	...	...	...	...	...
160	Venezuela	VEN	2018	71.9788	10709.9500	29825652	South America	NaN
161	Vietnam	VNM	2018	73.9757	6814.1420	94914328	Asia	NaN
162	Yemen	YEM	2018	64.5751	2284.8900	30790514	Asia	NaN
163	Zambia	ZMB	2018	62.3422	3534.0337	17835898	Africa	NaN
164	Zimbabwe	ZWE	2018	61.4141	1611.4052	15052191	Africa	NaN

165 rows x 8 columns

```
In [4]: print(df)
```

```

      Country Code  Year  LifeExpectancy  GDP per capita  Population \
0  Afghanistan  AFG  2018         63.0810        1934.5550    36686788
1    Albania   ALB  2018         79.1838       11104.1660     2877019
2    Algeria   DZA  2018         76.0656       14228.0250    41927008
3     Angola   AGO  2018         62.1438        7771.4420    31273538
4  Argentina   ARG  2018         76.9994       18556.3830    44413592
..      ...    ...    ...           ...           ...         ...
160  Venezuela  VEN  2018         71.9788       10709.9500    29825652
161   Vietnam  VNM  2018         73.9757        6814.1420    94914328
162    Yemen   YEM  2018         64.5751        2284.8900    30790514
163    Zambia  ZMB  2018         62.3422        3534.0337    17835898
164   Zimbabwe  ZWE  2018         61.4141        1611.4052    15052191

      Continent  PrintName
0         Asia         NaN
1        Europe         NaN
2         Africa         NaN
3         Africa         NaN
4  South America         NaN
..      ...    ...
160  South America         NaN
161         Asia         NaN
162         Asia         NaN
163         Africa         NaN
164         Africa         NaN

```

[165 rows x 8 columns]

The first pandas trick is `df.describe()`. This gives you all the basic stats. Remember how python works that `describe` is a function working on your `df`

```
In [5]: df.describe()
```

```
Out [5]:
```

	Year	LifeExpectancy	GDP per capita	Population	PrintName
count	165.0	165.000000	165.000000	1.650000e+02	13.0
mean	2018.0	72.631356	19053.786587	4.598704e+07	1.0
std	0.0	7.792401	20346.341739	1.575906e+08	0.0
min	2018.0	52.553600	623.488900	7.084000e+04	1.0
25%	2018.0	66.306400	4440.382000	4.270717e+06	1.0
50%	2018.0	73.975700	12165.795000	1.063328e+07	1.0
75%	2018.0	78.662200	27370.555000	3.239927e+07	1.0
max	2018.0	85.245600	153764.170000	1.417069e+09	1.0

But today we are just going to touch the surface of pandas and learn about files and then use the data.
First goal.

Read Data as CSV Comma Seperated Value

1. Save excel file as csv.
2. open in excel. NOT numbers
3. save as ms-dos csv
4. open with notes or text edit. What does it look like?
5. now open with python

```
In [6]: df=pd.read_csv('GDP_LifeExpectancy2018.csv')
```

```
In [7]: df
```

```
Out [7]:
```

	Country	Code	Year	LifeExpectancy	GDP per capita	Population	Continent	PrintName
0	Afghanistan	AFG	2018	63.0810	1934.5550	36686788	Asia	NaN
1	Albania	ALB	2018	79.1838	11104.1660	2877019	Europe	NaN
2	Algeria	DZA	2018	76.0656	14228.0250	41927008	Africa	NaN
3	Angola	AGO	2018	62.1438	7771.4420	31273538	Africa	NaN
4	Argentina	ARG	2018	76.9994	18556.3830	44413592	South America	NaN
...	...	...	...	...	...	...	...	...
160	Venezuela	VEN	2018	71.9788	10709.9500	29825652	South America	NaN
161	Vietnam	VNM	2018	73.9757	6814.1420	94914328	Asia	NaN
162	Yemen	YEM	2018	64.5751	2284.8900	30790514	Asia	NaN
163	Zambia	ZMB	2018	62.3422	3534.0337	17835898	Africa	NaN
164	Zimbabwe	ZWE	2018	61.4141	1611.4052	15052191	Africa	NaN

165 rows x 8 columns

```
In [8]: df.describe()
```

```
Out [8]:
```

	Year	LifeExpectancy	GDP per capita	Population
count	165.0	165.000000	165.000000	1.650000e+02
mean	2018.0	72.631356	19053.786587	4.598704e+07
std	0.0	7.792401	20346.341739	1.575906e+08
min	2018.0	52.553600	623.488900	7.084000e+04
25%	2018.0	66.306400	4440.382000	4.270717e+06
50%	2018.0	73.975700	12165.795000	1.063328e+07
75%	2018.0	78.662200	27370.555000	3.239927e+07
max	2018.0	85.245600	153764.170000	1.417069e+09

It is the same thing. You can use a csv file or an excel file. For large files csv are more efficient. But they both work. But now you know about csv and excel. lets read in excel. But also lets add sheet_name so you can read different sheets from an excel file.

```
In [9]: df=pd.read_excel('GDP_LifeExpectancy2018.xlsx',sheet_name='Sheet1')
```

Now to copy the graph we want

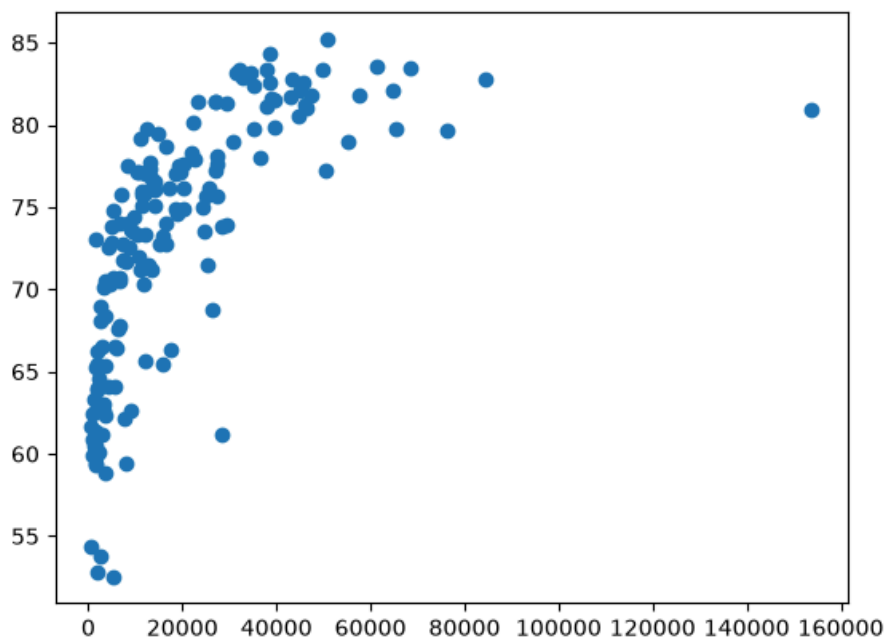
x=GDP

y=life expectancy

We will set them now to make things easier. In the future we can do it different ways

```
In [10]: x=df['GDP per capita']
y=df['LifeExpectancy']
```

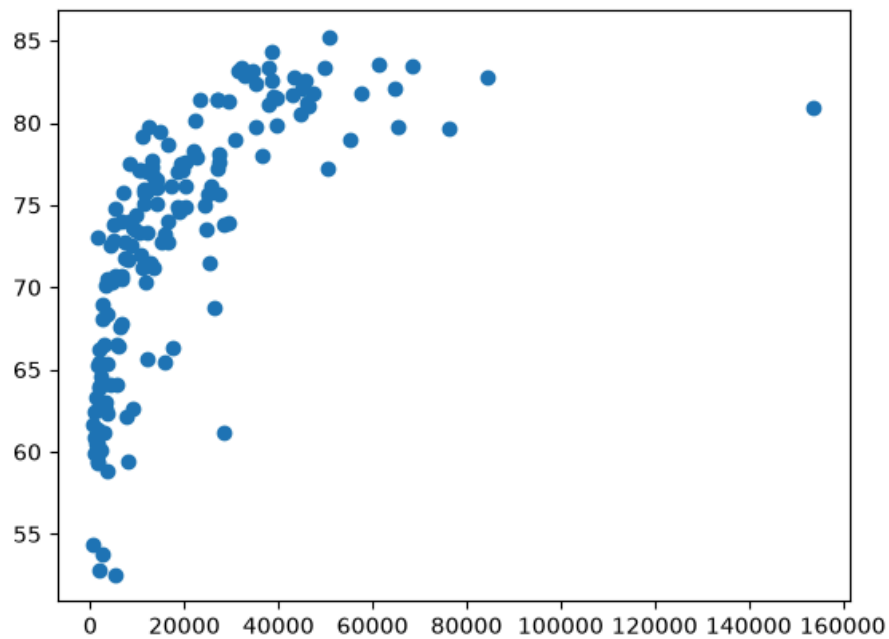
```
In [11]: fig,ax=plt.subplots()
ax.scatter(x,y)
plt.show()
```



You can also put the pandas columns right into scatter

```
In [12]: fig,ax=plt.subplots()
ax.scatter(df['GDP per capita'],df['LifeExpectancy'])
```

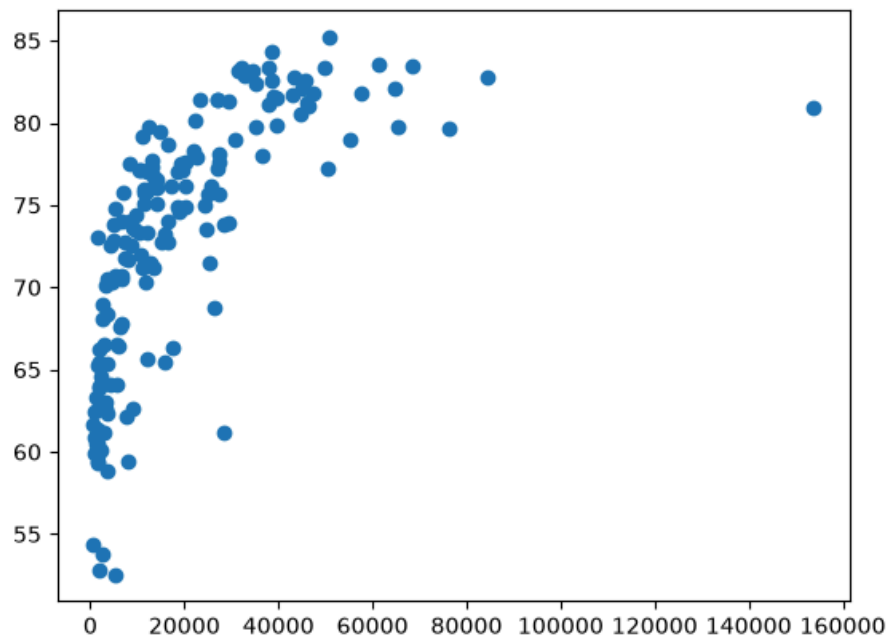
```
plt.show()
```



To keep things easier and clean for now lets set the x and y and make the plot all in one cell

```
In [13]: x=df['GDP per capita']
y=df['LifeExpectancy']

fig,ax=plt.subplots()
ax.scatter(x,y)
plt.show()
```



Then now let's see how a line fits through the data.

```
In [14]: stats.linregress(x,y)
```

```
Out[14]: LinregressResult(slope=0.0002549989236354205, intercept=67.77266129270281, rvalue=0.66581473498588
92, pvalue=1.7223524086251126e-22, stderr=2.2381961326783343e-05, intercept_stderr=0.6228910189586
092)
```

Now format it and name it like we did last class.

```
In [17]: LinResult = stats.linregress(x,y)
textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'\
        .format(LinResult.slope,LinResult.intercept
                ,LinResult.rvalue**2,LinResult.pvalue)
print(textstr)
```

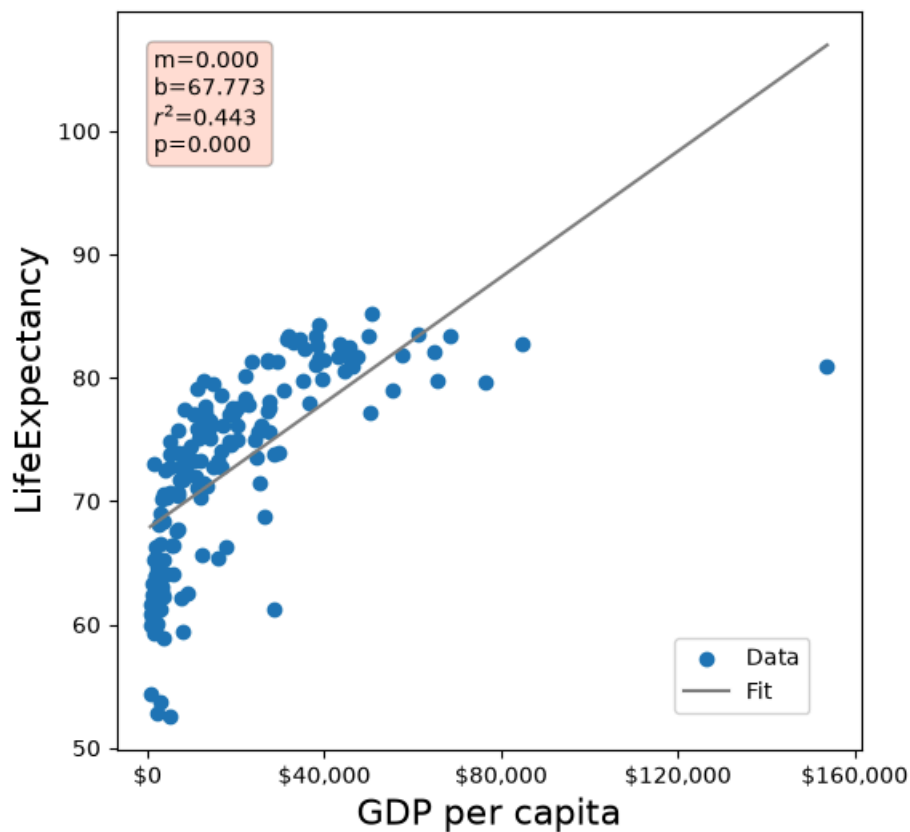
```
m=0.000
b=67.773
$r^2$=0.443
p=0.000
```

Can you add the line a nice box with the important data?

- On the `ax.set_xlabel` and `ylabel` I added the keyword `fontsize` to make the labels bigger.

```
In [20]:
```

```
Out[20]: <matplotlib.legend.Legend at 0x10ff6e050>
```



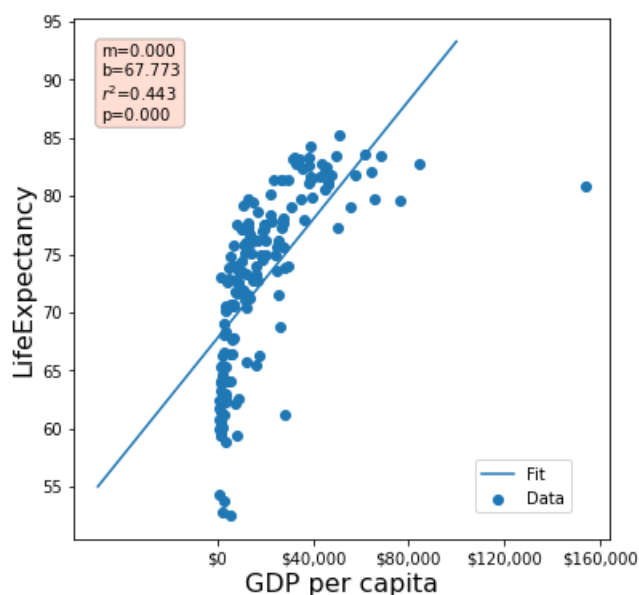
We need to talk x-fit and y-fit. The point of x-fit is to give a range over which you want to print your line. You control python and thus control the range of numbers in `x_fit`. Some options

1. just use `x`. this gives the exact dimensions and I am starting to do this more.
2. Use the min and max of `x`. it just gives two numbers so clean that way. But if you just use `x` you get the min and max
3. Just give it numbers you like. The downside is you need to keep resetting them. remember for this you need to pass `np.array([#1, #2])`

Go back to your graph and set `x_fit` to each of these bullets above and see if you get what you expect. My graph below is setting it to the numbers I wanted.

```
In [201...
```

Out[201]: <matplotlib.legend.Legend at 0x7fde1100b7c0>



The fit is bad!!!!

- p-value says the fit is statistically significant.
- But it doesn't capture the overall trend.
- I am not happy with the fit. The data is curved and our line is straight.
- Let's see what happens if we log the data. Remember in computer speak log is really ln or natural log and if you want log10 you need log10

```
In [28]: # get the data
x=np.log10(df['GDP per capita'])
y=df['LifeExpectancy']

# plot the data
fig,ax=plt.subplots()
fig.set_size_inches(6,6)
ax.scatter(x,y,label='Data')

ax.set_xlabel('Log 10 of GDP per capita',fontsize=16)
ax.set_ylabel('LifeExpectancy',fontsize=16)

#### Do the best fit line and plot it
LinResult = stats.linregress(x,y)

x_fit=np.linspace(min(x),max(x))
y_fit=x_fit*LinResult.slope+LinResult.intercept
ax.plot(x_fit,y_fit,label='Fit',color='gray')

# Use a comma in the x-axis with dollar signs
#ax.xaxis.set_major_formatter('${x:,}')

#make a larger step size on x-axis
#stepsize=40000
#ax.xaxis.set_ticks(np.arange(0, 160001, stepsize))

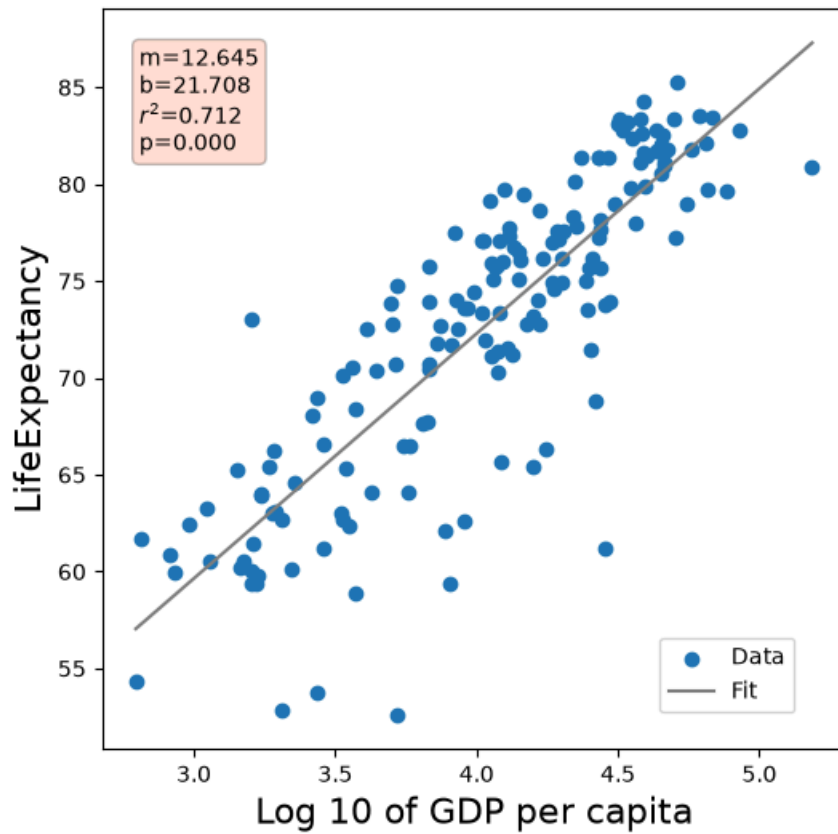
### Add the text box with fit data
props=dict(boxstyle='round',facecolor='coral',alpha=0.25)

textstr='m={:.3f}\nb={:.3f}\nr^2$={:.3f}\np={:.3f}'\
        .format(LinResult.slope,LinResult.intercept
                ,LinResult.rvalue**2,LinResult.pvalue)

ax.text(0.05,0.95,textstr,transform=ax.transAxes,fontsize=10\
        ,verticalalignment='top',bbox=props)
```

```
#Add the legend
ax.legend(loc=(.75,0.05))
```

Out[28]: <matplotlib.legend.Legend at 0x110334590>

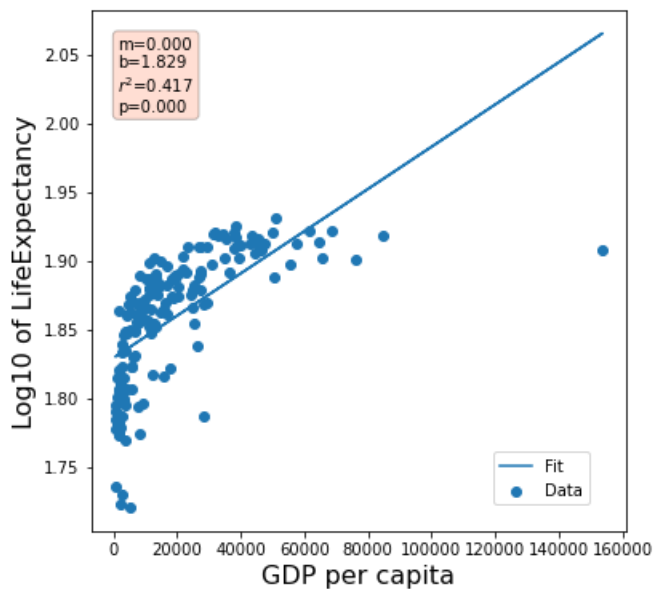


That is a great fit. Can you try doing

- Log10 of life Expectancy versus GDP
- Log10 of Life Expectancy versus Log10 of GDP

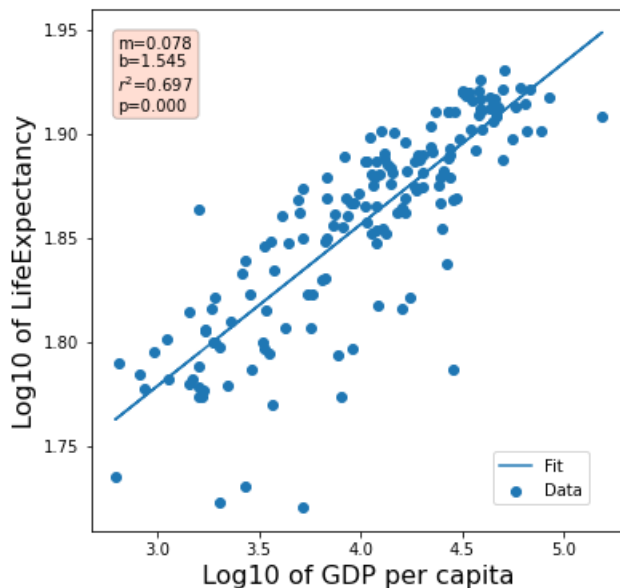
In [208...

Out[208]: <matplotlib.legend.Legend at 0x7fde2dca6520>



In [209]:

Out[209]: <matplotlib.legend.Legend at 0x7fde10716eb0>



The log10 of GDP per capita does give a nice fit!

Also if you want log paper you can use https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.loglog.html

https://matplotlib.org/stable/gallery/scales/log_demo.html

In [19]:

```
x=df['GDP per capita']
y=df['LifeExpectancy']

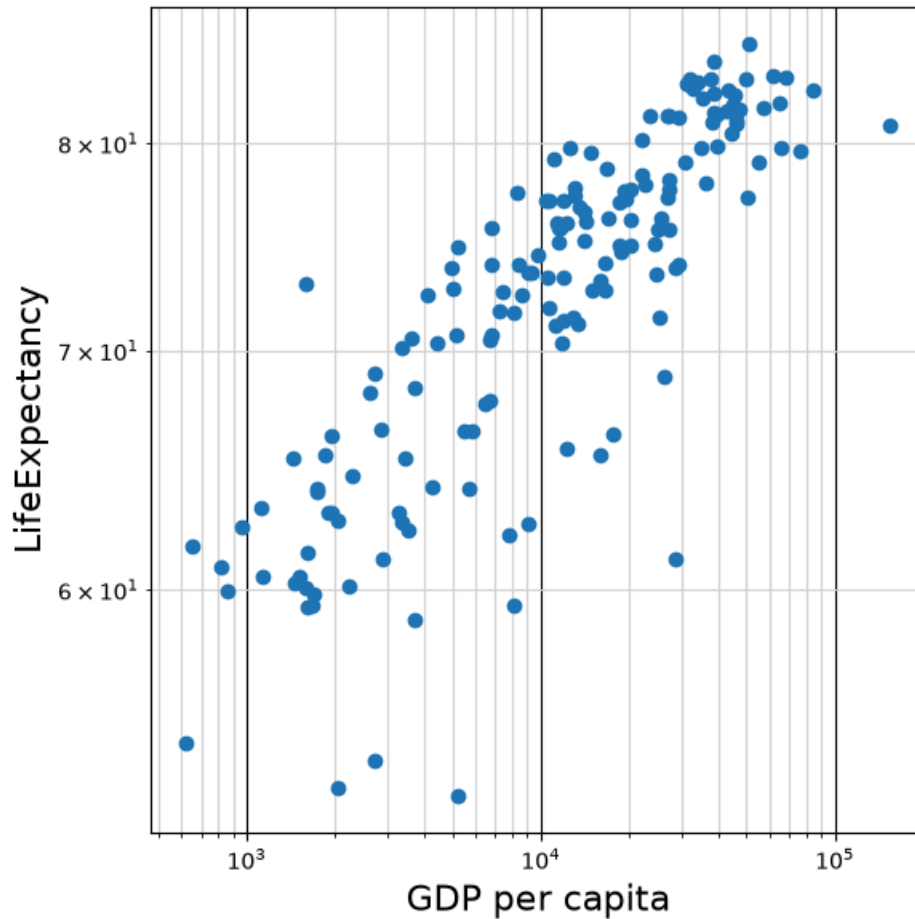
fig,ax=plt.subplots(layout='tight') #layout='tight' makes sure nothing is cutoff
fig.set_size_inches(6,6)

ax.loglog(x,y,linestyle='none',marker='o',base=10)

#This adds the major and minor grids and makes them slightly different colors
ax.grid(True,which="minor",ls="--", color='lightgrey')
ax.grid(True,which="major",ls="--", color='k')

ax.set_xlabel('GDP per capita',fontsize=16)
ax.set_ylabel('LifeExpectancy',fontsize=16)

plt.show()
```



To review today you should have read data in to a dataframe `df` with Pandas and then found a best fit line. We also looked at the log of the data to find a better fit. Then we plotted on log paper. Adding a line on log paper is annoying so we did not do that.

below I show you two more things. First I show you how I transformed the data from the website. Then I try to replicate the graph from the website.

In []:

Always Looat the data!

Anscombe's Quartet

- The point of this exercsie is to make you look at data
- Data can have the same statistics but be very different in the underlying structure
- Read in the file `Fits.xlsx`
- Each sheet `Sheet1`, `Sheet2`, `Sheet3`, `Sheet4`, `Sheet5` has very different shapes
- Plot each sheet
- Compare the statistics of each sheet. Are they similar?
- I started the code. Use your code from above to finish.

```
In [30]: # read the data
df=pd.read_excel('Fits.xlsx',sheet_name='Sheet1')

# get the data
x=df['X']
y=df['Y']

# plot the data
```

```

fig,ax=plt.subplots()
fig.set_size_inches(6,6)
ax.scatter(x,y,label='Data')

ax.set_xlabel('X',fontsize=16)
ax.set_ylabel('Y',fontsize=16)

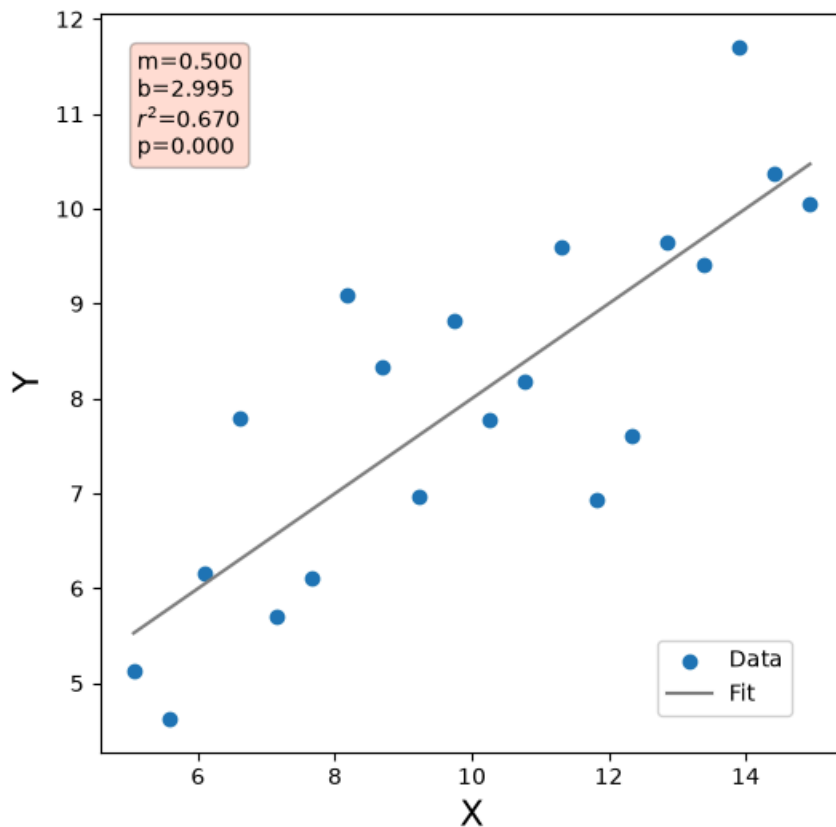
#### Do the best fit line and plot it

### Add the text box with fit data

#Add the legend

```

Out[30]: <matplotlib.legend.Legend at 0x110421750>



In []:

Use Gemini to replicate online graph

- Can we have AI make the same graph as online
- Use Geminin Thinking 3.6
- Here are my directions.

Using the attached file GDP_LifeExpectancy2018.csv can we make a plot of GDP versus life expectancy.

- Have the size of the scatter points scalue with population
- Color the points by continent
- Label the points with the name of the Country if PringName==TRUE
- Add legends for the continents and the size of the points

- Add dollar signs to the GDP values on the x-axis
- use `fig,ax=plt.subplots` nomenclature
- only use matplotlib
- I am running Python 3.11.15 with Matplotlib 3.11.1 in JupyterLab 4.6.3 on macOS. Please write code compatible with these versions

See if you get a similar answer or need to give more prompts. I deleted my code. I would try and move the continent legend so it doesn't block points. Also, you can upload a photo and just say copy with this data.

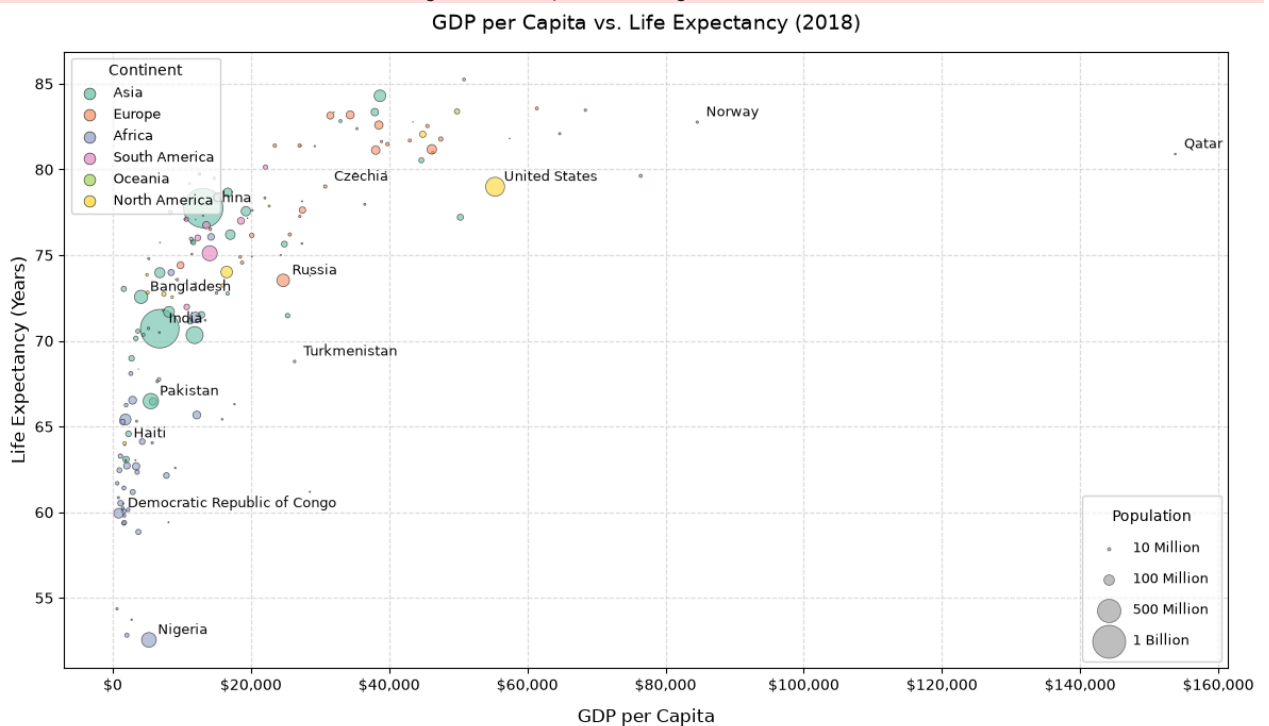
```
In [44]: import sys
import matplotlib
import jupyterlab

print("Python version: ", sys.version.split()[0])
print("Matplotlib version:", matplotlib.__version__)
print("JupyterLab version:", jupyterlab.__version__)
```

Python version: 3.11.15
Matplotlib version: 3.11.1
JupyterLab version: 4.6.3

In [45]:

findfont: Failed to find font weight medium, now using 400.



In []:

In []:

Answers

```
In [19]: # get the data
x=df['GDP per capita']
y=df['LifeExpectancy']

# plot the data
fig,ax=plt.subplots()
fig.set_size_inches(6,6)
```

```

ax.scatter(x,y,label='Data')

ax.set_xlabel('GDP per capita',fontsize=16)
ax.set_ylabel('LifeExpectancy',fontsize=16)

#### Do the best fit line and plot it
LinResult = stats.linregress(x,y)

x_fit=np.linspace(min(x),max(x))
y_fit=x_fit*LinResult.slope+LinResult.intercept
ax.plot(x_fit,y_fit,label='Fit',color='gray')

# Use a comma in the x-axis with dollar signs
ax.xaxis.set_major_formatter('${x:,}')

#make a larger step size on x-axis
stepsize=40000
ax.xaxis.set_ticks(np.arange(0, 160001, stepsize))

### Add the text box with fit data
props=dict(boxstyle='round',facecolor='coral',alpha=0.25)

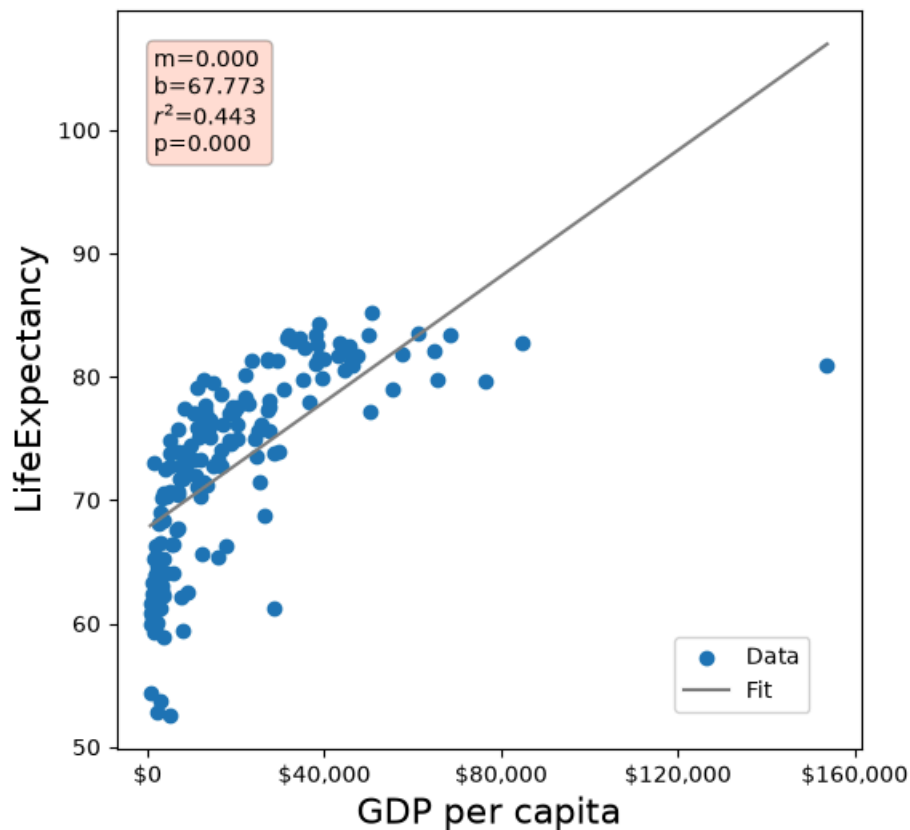
textstr='m={:.3f}\nb={:.3f}\nr^2$={:.3f}\np$={:.3f}'\
        .format(LinResult.slope,LinResult.intercept
                ,LinResult.rvalue**2,LinResult.pvalue)

ax.text(0.05,0.95,textstr,transform=ax.transAxes,fontsize=10\
        ,verticalalignment='top',bbox=props)

#Add the legend
ax.legend(loc=(.75,0.05))

```

Out[19]: <matplotlib.legend.Legend at 0x10feedb10>



Choosing my own x-fit

```

In [38]: # re-reading data
df=pd.read_excel('GDP_LifeExpectancy2018.xlsx')

# get the data
x=df['GDP per capita']
y=df['LifeExpectancy']

# plot the data
fig,ax=plt.subplots()
fig.set_size_inches(6,6)
ax.scatter(x,y, label='Data')

ax.set_xlabel('GDP per capita',fontsize=16)
ax.set_ylabel('LifeExpectancy',fontsize=16)

#### Do the best fit line and plot it
LinResult = stats.linregress(x,y)

x_fit=np.array([-1000,100000])
y_fit=x_fit*LinResult.slope+LinResult.intercept
ax.plot(x_fit,y_fit, label='Fit', color='gray')

# Use a comma in the x-axis with dollar signs
ax.xaxis.set_major_formatter('${x:,}')

#make a larger step size on x-axis
stepsize=40000
ax.xaxis.set_ticks(np.arange(0, 160001, stepsize))

### Add the text box with fit data
props=dict(boxstyle='round', facecolor='coral', alpha=0.25)

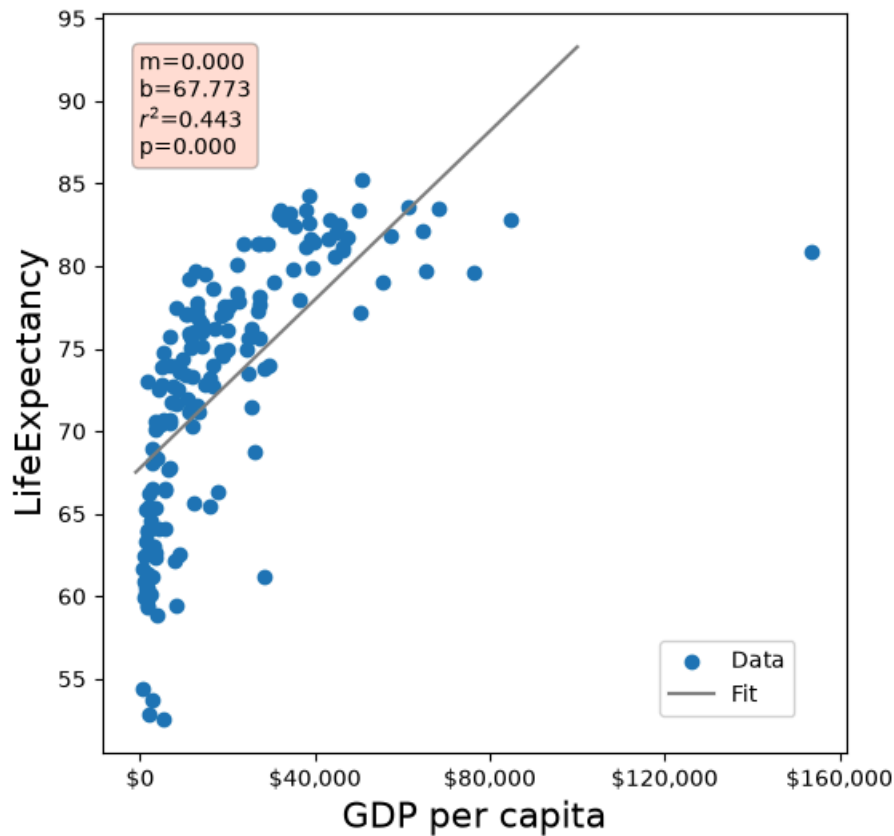
textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'\
        .format(LinResult.slope,LinResult.intercept
        ,LinResult.rvalue**2,LinResult.pvalue)

ax.text(0.05,0.95,textstr,transform=ax.transAxes,fontsize=10\
        ,verticalalignment='top',bbox=props)

#Add the legend
ax.legend(loc=(.75,0.05))

```

Out[38]: <matplotlib.legend.Legend at 0x1105baf10>



Adding log10

```
In [40]: # get the data
x=np.log10(df['GDP per capita'])
y=np.log10(df['LifeExpectancy'])

# plot the data
fig,ax=plt.subplots()
fig.set_size_inches(6,6)
ax.scatter(x,y,label='Data')

ax.set_xlabel('Log10 of GDP per capita',fontsize=16)
ax.set_ylabel('Log10 of LifeExpectancy',fontsize=16)

#### Do the best fit line and plot it
LinResult = stats.linregress(x,y)

x_fit=x
y_fit=x_fit*LinResult.slope+LinResult.intercept
ax.plot(x_fit,y_fit,label='Fit')

# Use a comma in the x-axis for dollars
#ax.xaxis.set_major_formatter('${x:,}')

#make a larger step size on x-axis
#ax.xaxis.set_ticks(np.arange(0, 160001, stepsize))

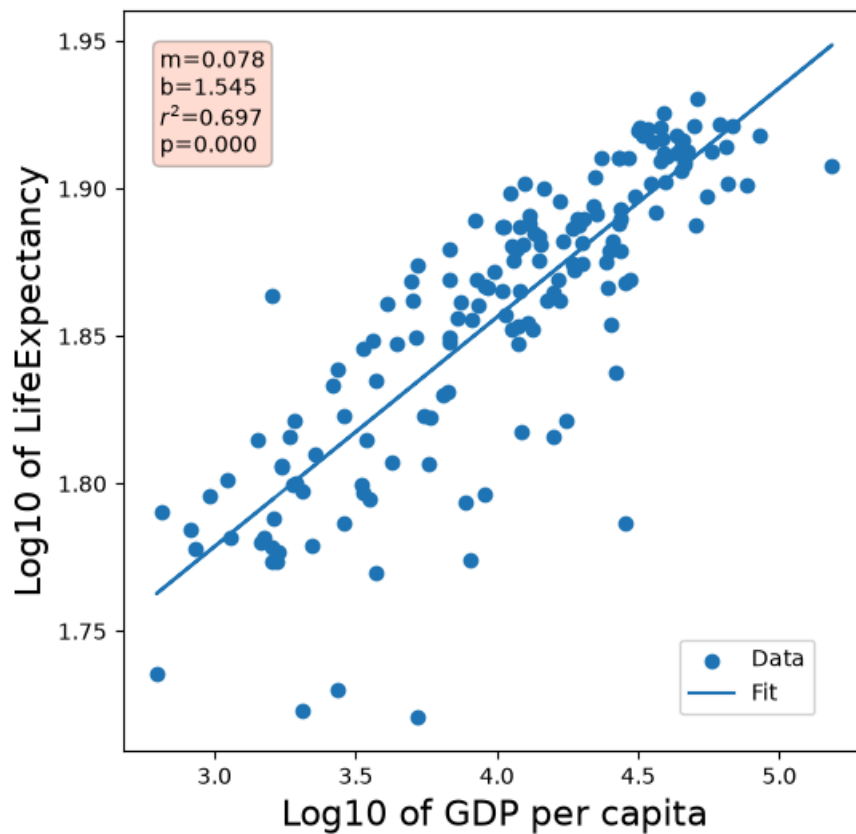
### Add the text box with fit data
props=dict(boxstyle='round',facecolor='coral',alpha=0.25)

textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\n$p$={:.3f}'\
        .format(LinResult.slope,LinResult.intercept
                ,LinResult.rvalue**2,LinResult.pvalue)

ax.text(0.05,0.95,textstr,transform=ax.transAxes,fontsize=10\
        ,verticalalignment='top',bbox=props)
```

```
#Add the legend
ax.legend(loc=(.75,0.05))
```

Out[40]: <matplotlib.legend.Legend at 0x11124c590>



Anscombe's Quartet

```
In [36]: # read the data
df=pd.read_excel('Fits.xlsx',sheet_name='Sheet1')

# get the data
x=df['X']
y=df['Y']

# plot the data
fig,ax=plt.subplots()
fig.set_size_inches(6,6)
ax.scatter(x,y,label='Data')

ax.set_xlabel('X',fontsize=16)
ax.set_ylabel('Y',fontsize=16)

#### Do the best fit line and plot it
LinResult = stats.linregress(x,y)

x_fit=np.linspace(min(x),max(x))
y_fit=x_fit*LinResult.slope+LinResult.intercept
ax.plot(x_fit,y_fit,label='Fit',color='gray')

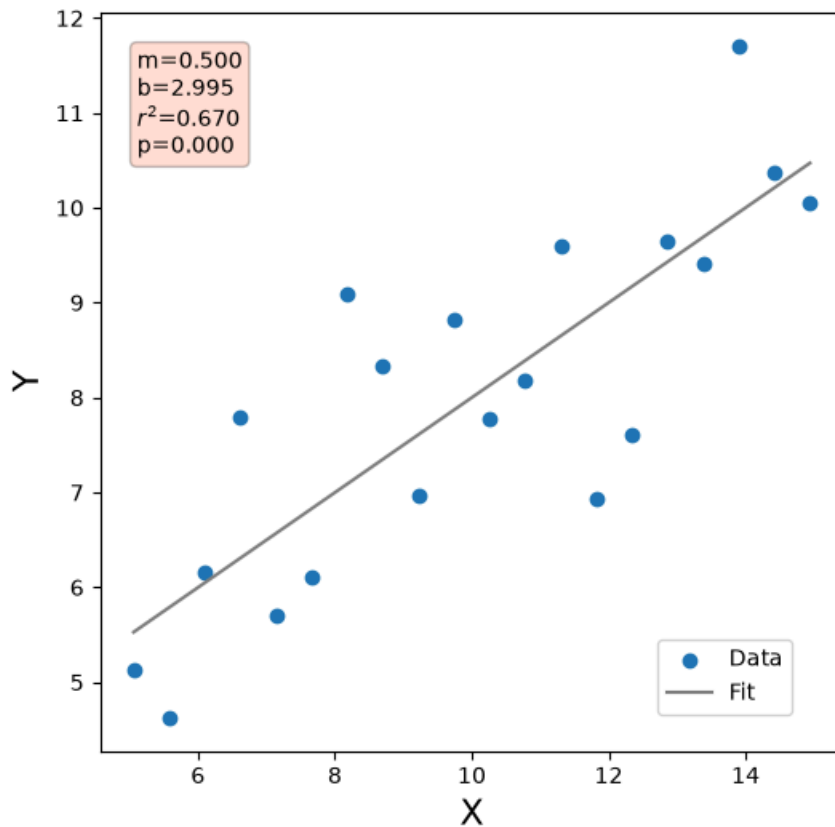
### Add the text box with fit data
props=dict(boxstyle='round',facecolor='coral',alpha=0.25)

textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\n$p$={:.3f}\n'
        .format(LinResult.slope,LinResult.intercept
                ,LinResult.rvalue**2,LinResult.pvalue)
```

```
ax.text(0.05,0.95,textstr,transform=ax.transAxes,fontsize=10\
        ,verticalalignment='top',bbox=props)

#Add the legend
ax.legend(loc=(.75,0.05))
```

Out[36]: <matplotlib.legend.Legend at 0x1104faf10>



In []:

Use Gemini to replicate online graph

- Can we have AI make the same graph as online
- Use Geminin Thinking 3.6
- Here are my directions.

Using the attached file GDP_LifeExpectancy2018.csv can we make a plot of GDP versus life expectancy.

- Have the size of the scatter points scalue with population
- Color the points by continet
- Label the points with the name of the Country if PringName==TRUE
- Add legends for the continents and the size of the points
- Add dollar signs to the GDP values on the x-axis
- use fig,ax=plt.subplots nomenclature
- only use matplotlib
- I am running Python 3.11.15 with Matplotlib 3.11.1 in JupyterLab 4.6.3 on macOS. Please write code compatible with these versions

See if you get a similar answer or need to give more prompts. I deleted my code.

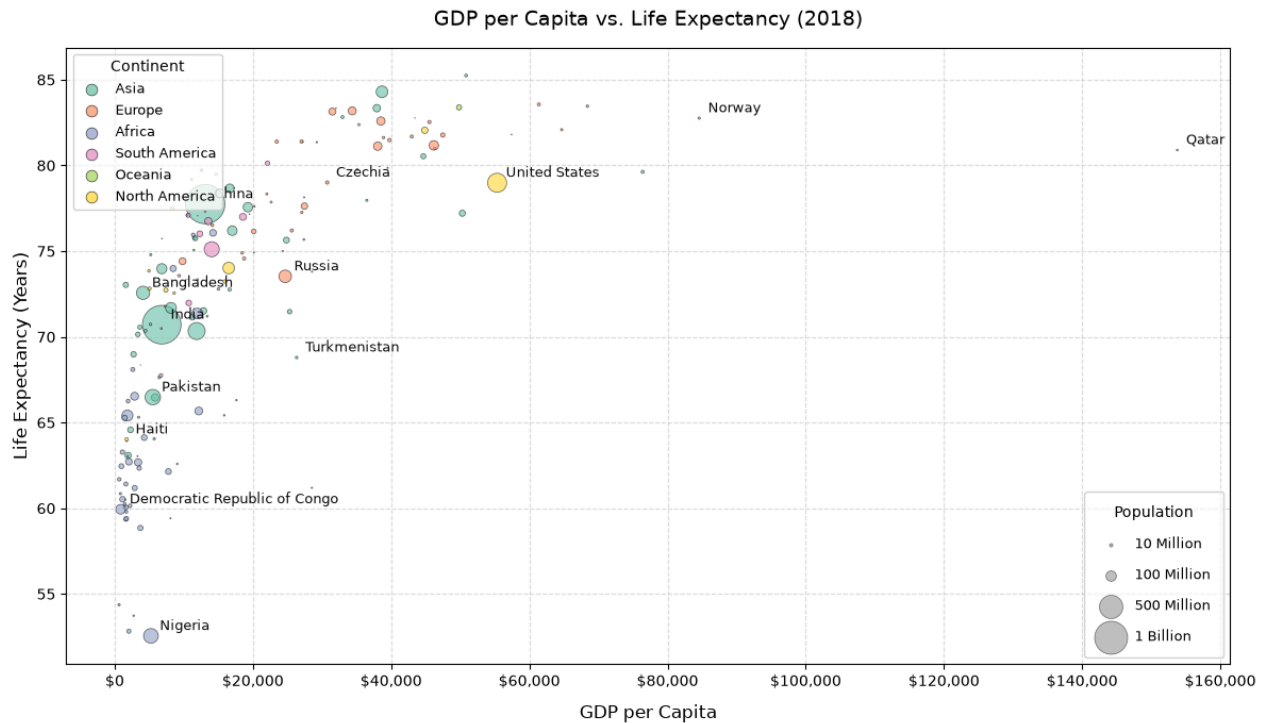
```
In [44]: import sys
import matplotlib
import jupyterlab
```

```
print("Python version: ", sys.version.split()[0])
print("Matplotlib version:", matplotlib.__version__)
print("JupyterLab version:", jupyterlab.__version__)
```

Python version: 3.11.15
 Matplotlib version: 3.11.1
 JupyterLab version: 4.6.3

In [45]:

findfont: Failed to find font weight medium, now using 400.



In []:

In []:

In []:

Extra Part 1

How I cleaned the data

If you want to know how I took the data from the website to our class all the data is below. I downloaded the raw data <https://ourworldindata.org/grapher/life-expectancy-vs-gdp-per-capita?xScale=linear>

First I read it in and printed the column names

```
In [21]: df=pd.read_csv('life-expectancy-vs-gdp-per-capita.csv')
```

```
In [22]: df.columns
```

```
Out[22]: Index(['Entity', 'Code', 'Year',
               'Period life expectancy at birth - Sex: all - Age: 0', 'GDP per capita',
               '417485-annotations', 'Population (historical estimates)', 'Continent'],
              dtype='object')
```

Renamed columns to sometime easier

```
In [23]: df.rename(columns={'Entity': 'Country',
                        , 'Period life expectancy at birth - Sex: all - Age: 0': 'LifeExpectancy'
                        , 'Population (historical estimates)': 'Population'
                        },
                  inplace=True)
```

```
In [24]: df.columns
```

```
Out[24]: Index(['Country', 'Code', 'Year', 'LifeExpectancy', 'GDP per capita',
               '417485-annotations', 'Population', 'Continent'],
              dtype='object')
```

I took years 2018 and 2015. 2015 has the continent and 2018 is the last year with all the data

```
In [25]: df=df[(df.Year==2018)|(df.Year==2015)]
```

I group the two lines into one

```
In [26]: agg_f={'Code':'last'
              , 'Year':'last'
              , 'LifeExpectancy':'last'
              , 'GDP per capita':'last'
              , '417485-annotations':'last'
              , 'Population':'last'
              , 'Continent':'first'
              }
dfG=df.groupby(['Country']).agg(agg_f)
```

I drop missing data

```
In [27]: dfG.dropna(subset=['LifeExpectancy',
                          , 'GDP per capita'
                          , 'Continent'],inplace=True)
```

Get rid of the weird column and save to excel

```
In [28]: dfG.drop('417485-annotations',axis=1,inplace=True)
```

```
In [29]: dfG.to_excel('GDP_LifeExpectancy2018_original.xlsx')
```

```
In [ ]:
```

Extra Part 2

Make Dataset with Gemini help looking at fits for Anscombe's Quartet

- Sheet1 - Linear
- Sheet2 - Quadratic
- Sheet3 - Outlier
- Sheet4 - High Leverage
- Sheet5 - Heteroscedastic
- "anscombe_5_tabs_20_points.xlsx"

```
In [30]: import pandas as pd

datasets = {
    "Sheet1": {
        "X": [5.06, 5.58, 6.10, 6.62, 7.14, 7.66, 8.18, 8.70, 9.22, 9.74, 10.26, 10.78, 11.30, 11.82],
        "Y": [5.13, 4.63, 6.15, 7.80, 5.70, 6.10, 9.09, 8.33, 6.97, 8.82, 7.78, 8.18, 9.59, 6.93, 7.26],
    },
    "Sheet2": {
        "X": [5.06, 5.58, 6.10, 6.62, 7.14, 7.66, 8.18, 8.70, 9.22, 9.74, 10.26, 10.78, 11.30, 11.82],
        "Y": [5.13, 4.63, 6.15, 7.80, 5.70, 6.10, 9.09, 8.33, 6.97, 8.82, 7.78, 8.18, 9.59, 6.93, 7.26],
    },
}
```

```

        "Y": [3.50, 4.40, 5.23, 5.99, 6.68, 7.29, 7.84, 8.31, 8.71, 9.04, 9.30, 9.49, 9.61, 9.66, 9.68, 9.71, 9.74, 9.77, 9.80, 9.83, 9.86, 9.89, 9.92, 9.95, 9.98, 10.01, 10.04, 10.07, 10.10, 10.13, 10.16, 10.19, 10.22, 10.25, 10.28, 10.31, 10.34, 10.37, 10.40, 10.43, 10.46, 10.49, 10.52, 10.55, 10.58, 10.61, 10.64, 10.67, 10.70, 10.73, 10.76, 10.79, 10.82, 10.85, 10.88, 10.91, 10.94, 10.97, 11.00, 11.03, 11.06, 11.09, 11.12, 11.15, 11.18, 11.21, 11.24, 11.27, 11.30, 11.33, 11.36, 11.39, 11.42, 11.45, 11.48, 11.51, 11.54, 11.57, 11.60, 11.63, 11.66, 11.69, 11.72, 11.75, 11.78, 11.81, 11.84, 11.87, 11.90, 11.93, 11.96, 11.99, 12.02, 12.05, 12.08, 12.11, 12.14, 12.17, 12.20, 12.23, 12.26, 12.29, 12.32, 12.35, 12.38, 12.41, 12.44, 12.47, 12.50, 12.53, 12.56, 12.59, 12.62, 12.65, 12.68, 12.71, 12.74, 12.77, 12.80, 12.83, 12.86, 12.89, 12.92, 12.95, 12.98, 13.01, 13.04, 13.07, 13.10, 13.13, 13.16, 13.19, 13.22, 13.25, 13.28, 13.31, 13.34, 13.37, 13.40, 13.43, 13.46, 13.49, 13.52, 13.55, 13.58, 13.61, 13.64, 13.67, 13.70, 13.73, 13.76, 13.79, 13.82, 13.85, 13.88, 13.91, 13.94, 13.97, 14.00, 14.03, 14.06, 14.09, 14.12, 14.15, 14.18, 14.21, 14.24, 14.27, 14.30, 14.33, 14.36, 14.39, 14.42, 14.45, 14.48, 14.51, 14.54, 14.57, 14.60, 14.63, 14.66, 14.69, 14.72, 14.75, 14.78, 14.81, 14.84, 14.87, 14.90, 14.93, 14.96, 14.99, 15.02, 15.05, 15.08, 15.11, 15.14, 15.17, 15.20, 15.23, 15.26, 15.29, 15.32, 15.35, 15.38, 15.41, 15.44, 15.47, 15.50, 15.53, 15.56, 15.59, 15.62, 15.65, 15.68, 15.71, 15.74, 15.77, 15.80, 15.83, 15.86, 15.89, 15.92, 15.95, 15.98, 16.01, 16.04, 16.07, 16.10, 16.13, 16.16, 16.19, 16.22, 16.25, 16.28, 16.31, 16.34, 16.37, 16.40, 16.43, 16.46, 16.49, 16.52, 16.55, 16.58, 16.61, 16.64, 16.67, 16.70, 16.73, 16.76, 16.79, 16.82, 16.85, 16.88, 16.91, 16.94, 16.97, 17.00, 17.03, 17.06, 17.09, 17.12, 17.15, 17.18, 17.21, 17.24, 17.27, 17.30, 17.33, 17.36, 17.39, 17.42, 17.45, 17.48, 17.51, 17.54, 17.57, 17.60, 17.63, 17.66, 17.69, 17.72, 17.75, 17.78, 17.81, 17.84, 17.87, 17.90, 17.93, 17.96, 17.99, 18.02, 18.05, 18.08, 18.11, 18.14, 18.17, 18.20, 18.23, 18.26, 18.29, 18.32, 18.35, 18.38, 18.41, 18.44, 18.47, 18.50, 18.53, 18.56, 18.59, 18.62, 18.65, 18.68, 18.71, 18.74, 18.77, 18.80, 18.83, 18.86, 18.89, 18.92, 18.95, 18.98, 19.01, 19.04, 19.07, 19.10, 19.13, 19.16, 19.19, 19.22, 19.25, 19.28, 19.31, 19.34, 19.37, 19.40, 19.43, 19.46, 19.49, 19.52, 19.55, 19.58, 19.61, 19.64, 19.67, 19.70, 19.73, 19.76, 19.79, 19.82, 19.85, 19.88, 19.91, 19.94, 19.97, 20.00, 20.03, 20.06, 20.09, 20.12, 20.15, 20.18, 20.21, 20.24, 20.27, 20.30, 20.33, 20.36, 20.39, 20.42, 20.45, 20.48, 20.51, 20.54, 20.57, 20.60, 20.63, 20.66, 20.69, 20.72, 20.75, 20.78, 20.81, 20.84, 20.87, 20.90, 20.93, 20.96, 20.99, 21.02, 21.05, 21.08, 21.11, 21.14, 21.17, 21.20, 21.23, 21.26, 21.29, 21.32, 21.35, 21.38, 21.41, 21.44, 21.47, 21.50, 21.53, 21.56, 21.59, 21.62, 21.65, 21.68, 21.71, 21.74, 21.77, 21.80, 21.83, 21.86, 21.89, 21.92, 21.95, 21.98, 22.01, 22.04, 22.07, 22.10, 22.13, 22.16, 22.19, 22.22, 22.25, 22.28, 22.31, 22.34, 22.37, 22.40, 22.43, 22.46, 22.49, 22.52, 22.55, 22.58, 22.61, 22.64, 22.67, 22.70, 22.73, 22.76, 22.79, 22.82, 22.85, 22.88, 22.91, 22.94, 22.97, 23.00, 23.03, 23.06, 23.09, 23.12, 23.15, 23.18, 23.21, 23.24, 23.27, 23.30, 23.33, 23.36, 23.39, 23.42, 23.45, 23.48, 23.51, 23.54, 23.57, 23.60, 23.63, 23.66, 23.69, 23.72, 23.75, 23.78, 23.81, 23.84, 23.87, 23.90, 23.93, 23.96, 23.99, 24.02, 24.05, 24.08, 24.11, 24.14, 24.17, 24.20, 24.23, 24.26, 24.29, 24.32, 24.35, 24.38, 24.41, 24.44, 24.47, 24.50, 24.53, 24.56, 24.59, 24.62, 24.65, 24.68, 24.71, 24.74, 24.77, 24.80, 24.83, 24.86, 24.89, 24.92, 24.95, 24.98, 25.01, 25.04, 25.07, 25.10, 25.13, 25.16, 25.19, 25.22, 25.25, 25.28, 25.31, 25.34, 25.37, 25.40, 25.43, 25.46, 25.49, 25.52, 25.55, 25.58, 25.61, 25.64, 25.67, 25.70, 25.73, 25.76, 25.79, 25.82, 25.85, 25.88, 25.91, 25.94, 25.97, 26.00, 26.03, 26.06, 26.09, 26.12, 26.15, 26.18, 26.21, 26.24, 26.27, 26.30, 26.33, 26.36, 26.39, 26.42, 26.45, 26.48, 26.51, 26.54, 26.57, 26.60, 26.63, 26.66, 26.69, 26.72, 26.75, 26.78, 26.81, 26.84, 26.87, 26.90, 26.93, 26.96, 26.99, 27.02, 27.05, 27.08, 27.11, 27.14, 27.17, 27.20, 27.23, 27.26, 27.29, 27.32, 27.35, 27.38, 27.41, 27.44, 27.47, 27.50, 27.53, 27.56, 27.59, 27.62, 27.65, 27.68, 27.71, 27.74, 27.77, 27.80, 27.83, 27.86, 27.89, 27.92, 27.95, 27.98, 28.01, 28.04, 28.07, 28.10, 28.13, 28.16, 28.19, 28.22, 28.25, 28.28, 28.31, 28.34, 28.37, 28.40, 28.43, 28.46, 28.49, 28.52, 28.55, 28.58, 28.61, 28.64, 28.67, 28.70, 28.73, 28.76, 28.79, 28.82, 28.85, 28.88, 28.91, 28.94, 28.97, 29.00, 29.03, 29.06, 29.09, 29.12, 29.15, 29.18, 29.21, 29.24, 29.27, 29.30, 29.33, 29.36, 29.39, 29.42, 29.45, 29.48, 29.51, 29.54, 29.57, 29.60, 29.63, 29.66, 29.69, 29.72, 29.75, 29.78, 29.81, 29.84, 29.87, 29.90, 29.93, 29.96, 29.99, 30.02, 30.05, 30.08, 30.11, 30.14, 30.17, 30.20, 30.23, 30.26, 30.29, 30.32, 30.35, 30.38, 30.41, 30.44, 30.47, 30.50, 30.53, 30.56, 30.59, 30.62, 30.65, 30.68, 30.71, 30.74, 30.77, 30.80, 30.83, 30.86, 30.89, 30.92, 30.95, 30.98, 31.01, 31.04, 31.07, 31.10, 31.13, 31.16, 31.19, 31.22, 31.25, 31.28, 31.31, 31.34, 31.37, 31.40, 31.43, 31.46, 31.49, 31.52, 31.55, 31.58, 31.61, 31.64, 31.67, 31.70, 31.73, 31.76, 31.79, 31.82, 31.85, 31.88, 31.91, 31.94, 31.97, 32.00, 32.03, 32.06, 32.09, 32.12, 32.15, 32.18, 32.21, 32.24, 32.27, 32.30, 32.33, 32.36, 32.39, 32.42, 32.45, 32.48, 32.51, 32.54, 32.57, 32.60, 32.63, 32.66, 32.69, 32.72, 32.75, 32.78, 32.81, 32.84, 32.87, 32.90, 32.93, 32.96, 32.99, 33.02, 33.05, 33.08, 33.11, 33.14, 33.17, 33.20, 33.23, 33.26, 33.29, 33.32, 33.35, 33.38, 33.41, 33.44, 33.47, 33.50, 33.53, 33.56, 33.59, 33.62, 33.65, 33.68, 33.71, 33.74, 33.77, 33.80, 33.83, 33.86, 33.89, 33.92, 33.95, 33.98, 34.01, 34.04, 34.07, 34.10, 34.13, 34.16, 34.19, 34.22, 34.25, 34.28, 34.31, 34.34, 34.37, 34.40, 34.43, 34.46, 34.49, 34.52, 34.55, 34.58, 34.61, 34.64, 34.67, 34.70, 34.73, 34.76, 34.79, 34.82, 34.85, 34.88, 34.91, 34.94, 34.97, 35.00, 35.03, 35.06, 35.09, 35.12, 35.15, 35.18, 35.21, 35.24, 35.27, 35.30, 35.33, 35.36, 35.39, 35.42, 35.45, 35.48, 35.51, 35.54, 35.57, 35.60, 35.63, 35.66, 35.69, 35.72, 35.75, 35.78, 35.81, 35.84, 35.87, 35.90, 35.93, 35.96, 35.99, 36.02, 36.05, 36.08, 36.11, 36.14, 36.17, 36.20, 36.23, 36.26, 36.29, 36.32, 36.35, 36.38, 36.41, 36.44, 36.47, 36.50, 36.53, 36.56, 36.59, 36.62, 36.65, 36.68, 36.71, 36.74, 36.77, 36.80, 36.83, 36.86, 36.89, 36.92, 36.95, 36.98, 37.01, 37.04, 37.07, 37.10, 37.13, 37.16, 37.19, 37.22, 37.25, 37.28, 37.31, 37.34, 37.37, 37.40, 37.43, 37.46, 37.49, 37.52, 37.55, 37.58, 37.61, 37.64, 37.67, 37.70, 37.73, 37.76, 37.79, 37.82, 37.85, 37.88, 37.91, 37.94, 37.97, 38.00, 38.03, 38.06, 38.09, 38.12, 38.15, 38.18, 38.21, 38.24, 38.27, 38.30, 38.33, 38.36, 38.39, 38.42, 38.45, 38.48, 38.51, 38.54, 38.57, 38.60, 38.63, 38.66, 38.69, 38.72, 38.75, 38.78, 38.81, 38.84, 38.87, 38.90, 38.93, 38.96, 38.99, 39.02, 39.05, 39.08, 39.11, 39.14, 39.17, 39.20, 39.23, 39.26, 39.29, 39.32, 39.35, 39.38, 39.41, 39.44, 39.47, 39.50, 39.53, 39.56, 39.59, 39.62, 39.65, 39.68, 39.71, 39.74, 39.77, 39.80, 39.83, 39.86, 39.89, 39.92, 39.95, 39.98, 40.01, 40.04, 40.07, 40.10, 40.13, 40.16, 40.19, 40.22, 40.25, 40.28, 40.31, 40.34, 40.37, 40.40, 40.43, 40.46, 40.49, 40.52, 40.55, 40.58, 40.61, 40.64, 40.67, 40.70, 40.73, 40.76, 40.79, 40.82, 40.85, 40.88, 40.91, 40.94, 40.97, 41.00, 41.03, 41.06, 41.09, 41.12, 41.15, 41.18, 41.21, 41.24, 41.27, 41.30, 41.33, 41.36, 41.39, 41.42, 41.45, 41.48, 41.51, 41.54, 41.57, 41.60, 41.63, 41.66, 41.69, 41.72, 41.75, 41.78, 41.81, 41.84, 41.87, 41.90, 41.93, 41.96, 41.99, 42.02, 42.05, 42.08, 42.11, 42.14, 42.17, 42.20, 42.23, 42.26, 42.29, 42.32, 42.35, 42.38, 42.41, 42.44, 42.47, 42.50, 42.53, 42.56, 42.59, 42.62, 42.65, 42.68, 42.71, 42.74, 42.77, 42.80, 42.83, 42.86, 42.89, 42.92, 42.95, 42.98, 43.01, 43.04, 43.07, 43.10, 43.13, 43.16, 43.19, 43.22, 43.25, 43.28, 43.31, 43.34, 43.37, 43.40, 43.43, 43.46, 43.49, 43.52, 43.55, 43.58, 43.61, 43.64, 43.67, 43.70, 43.73, 43.76, 43.79, 43.82, 43.85, 43.88, 43.91, 43.94, 43.97, 44.00, 44.03, 44.06, 44.09, 44.12, 44.15, 44.18, 44.21, 44.24, 44.27, 44.30, 44.33, 44.36, 44.39, 44.42, 44.45, 44.48, 44.51, 44.54, 44.57, 44.60, 44.63, 44.66, 44.69, 44.72, 44.75, 44.78, 44.81, 44.84, 44.87, 44.90, 44.93, 44.96, 44.99, 45.02, 45.05, 45.08, 45.11, 45.14, 45.17, 45.20, 45.23, 45.26, 45.29, 45.32, 45.35, 45.38, 45.41, 45.44, 45.47, 45.50, 45.53, 45.56, 45.59, 45.62, 45.65, 45.68, 45.71, 45.74, 45.77, 45.80, 45.83, 45.86, 45.89, 45.92, 45.95, 45.98, 46.01, 46.04, 46.07, 46.10, 46.13, 46.16, 46.19, 46.22, 46.25, 46.28, 46.31, 46.34, 46.37, 46.40, 46.43, 46.46, 46.49, 46.52, 46.55, 46.58, 46.61, 46.64, 46.67, 46.70, 46.73, 46.76, 46.79, 46.82, 46.85, 46.88, 46.91, 46.94, 46.97, 47.00, 47.03, 47.06, 47.09, 47.12, 47.15, 47.18, 47.21, 47.24, 47.27, 47.30, 47.33, 47.36, 47.39, 47.42, 47.45, 47.48, 47.51, 47.54, 47.57, 47.60, 47.63, 47.66, 47.69, 47.72, 47.75, 47.78, 47.81, 47.84, 47.87, 47.90, 47.93, 47.96, 47.99, 48.02, 48.05, 48.08, 48.11, 48.14, 48.17, 48.20, 48.23, 48.26, 48.29, 48.32, 48.35, 48.38, 48.41, 48.44, 48.47, 48.50, 48.53, 48.56, 48.59, 48.62, 48.65, 48.68, 48.71, 48.74, 48.77, 48.80, 48.83, 48.86, 48.89, 48.92, 48.95, 48.98, 49.01, 49.04, 49.07, 49.10, 49.13, 49.16, 49.19, 49.22, 49.25, 49.28, 49.31, 49.34, 49.37, 49.40, 49.43, 49.46, 49.49, 49.52, 49.55, 49.58, 49.61, 49.64, 49.67, 49.70, 49.73, 49.76, 49.79, 49.82, 49.85, 49.88, 49.91, 49.94, 49.97, 50.00, 50.03, 50.06, 50.09, 50.12, 50.15, 50.18, 50.21, 50.24, 50.27, 50.30, 50.33, 50.36, 50.39, 50.42, 50.45, 50.48, 50.51, 50.54, 50.57, 50.60, 50.63, 50.66, 50.69, 50.72, 50.75, 50.78, 50.81, 50.84, 50.87, 50.90, 50.93, 50.96, 50.99, 51.02, 51.05, 51.08, 51.11, 51.14, 51.17, 51.20, 51.23, 51.26, 51.29, 51.32, 51.35, 51.38, 51.41, 51.44, 51.47, 51.50, 51.53, 51.56, 51.59, 51.62, 51.65, 51.68, 51.71, 51.74, 51.77, 51.80, 51.83, 51.86, 51.89, 51.92, 51.95, 51.98, 52.01, 52.04, 52.07, 52.10, 52.13, 52.16, 52.19, 52.22, 52.25, 52.28, 52.31, 52.34, 52.37, 52.40, 52.43, 52.46, 52.49, 52.52, 52.55, 52.58, 52.61, 52.64, 52.67, 52.70, 52.73, 52.76, 52.79, 52.82, 52.85, 52.88, 52.91, 52.94, 52.97, 53.00, 53.03, 53.06, 53.09, 53.12, 53.15, 53.18, 53.21, 53.24, 53.27, 53.30, 53.33, 53.36, 53.39, 53.42, 53.45, 53.48, 53.51, 53.54, 53.57, 53.60, 53.63, 53.66, 53.69, 53.72, 53.75, 53.78, 53.81, 53.84, 53.87, 53.90, 53.93, 53.96, 53.99, 54.02, 54.05, 54.08, 54.11, 54.14, 54.17, 54.20, 54.23, 54.26, 54.29, 54.32, 54.35, 54.38, 54.41, 54.44, 54.47, 54.50, 54.53, 54.56, 54.59, 54.62, 54.65, 54.68, 54.71, 54.74, 54.77, 54.80, 54.83, 54.86, 54.89, 54.92, 54.95, 54.98, 55.01, 55.04, 55.07, 55.10, 55.13, 55.16, 55.19, 55.22, 55.25, 55.28, 55.31, 55.34, 55.37, 55.40, 55.43, 55.46, 55.49, 55.52, 55.55, 55.58, 55.61, 55.64, 55.67, 55.70, 55.73, 55.76, 55.79, 55.82, 55.85, 55.88, 55.91, 55.94, 55.97, 56.00, 56.03, 56.06, 56.09, 56.12, 56.15, 56.18, 56.21, 56.24, 56.27, 56.30, 56.33, 56.36, 56.39, 56.42, 56.45, 56.48, 56.51, 56.54, 56.57, 56.60, 56.63, 56.66, 56.69, 56.72, 56.75, 56.78, 56.81, 56.84, 56.87, 56.90, 56.93, 56.96, 56.99, 57.02, 57.05, 57.08, 57.11, 57.14, 57.17, 57.20, 57.23, 57.26, 57.29, 57.32, 57.35, 57.38, 57.41, 57.44, 57.47, 57.50, 57.53, 57.56, 57.59, 57.62, 57.65, 57.68, 57.71, 57.74, 57.77, 57.80, 57.83, 57.86, 57.89, 57.92, 57.95, 57.98, 58.01, 58.04, 58.07, 58.10, 58.13, 58.16, 58.19, 58.22, 58.25, 58.28, 58.31, 58.34, 58.37, 58.40, 58.43, 58.46, 58.49, 58.52, 58.55, 58.58, 58.61, 58.64, 58.67, 58.70, 58.73, 58.76, 58.79, 58.82, 58.85, 58.88, 58.91, 58.94, 58.97, 59.00, 59.03, 59.06, 59.09, 59.12, 59.15, 59.18, 59.21, 59.24, 59.27, 59.30, 59.33, 59.36, 59.39, 59.42, 59.45, 59.48, 59.51, 59.54, 59.57, 59.60, 59.63, 59.66, 59.69, 59.72, 59.75, 59.78, 59.81, 59.84, 59.87, 59.90, 59.93, 59.96, 59.99, 60.02, 60.05, 60.08, 60.11, 60.14, 60.17, 60.20, 60.23, 60.26, 60.29, 60.32, 60.35, 60.38, 60.41, 60.44, 60.47, 60.50, 60.53, 60.56, 60.59, 60.62, 60.65, 60.68, 60.71, 60.74, 60.77, 60.80, 60.83, 60.86, 60.89, 60.92, 60.95, 60.98, 61.01, 61.04, 61.07, 61.10, 61.13, 61.16, 61.19, 61.22, 61.25, 61.28, 61.31, 61.34, 61.37, 61.40, 61.43, 61.46, 61.49, 61.52, 61.55, 61.58, 61.61, 61.64, 61.67, 61.70, 61.73, 61.76, 61.79, 61.82, 61.85, 61.88, 61.91, 61.94, 61.97, 62.00, 62.03, 62.06, 62.09, 62.12, 62.15, 62.18, 62.21, 62.24, 62.27,
```

Exit Ticket

Name:

uni:

Q1. Sketch two graphs with some scatter points and a best fit line. Make the graphs have similar r^2 but different distribution of points.

In []:

In []: